The National University of Computer and Emerging Sciences, Lahore

Course Outline

Compiler Construction CS-4031

Semester Fall-2026

Coordinator: Dr. Faisal Aslam

Office hours: Monday to Thursday 10:00 to 11:00

Credit Hours: 3

Prerequisites: Programming Fundamentals, Theory of Automata

Instructors: Farooq Ahmad, Faisal Aslam, Aamir Raheem, Hamad Ulqudous

Course Description

Compiler Construction introduces students to the theory and practice of translating high-level programs into executable code. The course follows a working compiler front-to-back: lexical analysis, syntax analysis (parsing), semantic analysis, intermediate-code generation, run-time environments, and code optimization. Emphasis is placed on the formal models that make each phase tractable – regular expressions and finite automata for scanning, context-free grammars and both top-down and bottom-up parsing (including the industry-standard LALR(1) algorithm) for parsing, and data-flow analysis for optimization – alongside hands-on construction of each phase using standard tools (Flex, Yacc/Bison) and a semester-long project. By the end of the course, students should be able to explain why each phase of a compiler is structured the way it is, analyze and choose appropriate parsing and optimization algorithms for a given language, and independently design and implement the lexical analyzer, parser, and intermediate-code generator for a small language.

Program Learning Outcomes (PLOs)

This course covers the following PLOs:

PLO#	PLO Name	PLO Description
PLO 2	Knowledge for Solving Computing Problems	Apply knowledge of computing fundamentals, knowledge of a computing specialization, and mathematics, science, and domain knowledge appropriate for the computing specialization to the abstraction and conceptualization of computing models from defined problems and requirements
PLO 3	Problem Analysis	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
PLO 4	Design/Development of Solutions	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations

PLO#	PLO Name	PLO Description
PLO 5	Modern Tool Usage	Focuses on creating, selecting, adapting, and applying modern tools, techniques, and resources to solve complex computing problems, understanding the limitations of these tools
PLO 6	Individual and Team Work	Aims to help students function effectively both as individuals and as team members or leaders in diverse and multidisciplinary settings

Course Learning Outcomes (CLOs)

CS-4031 is a core Computer Science course with Programming Fundamentals and Theory of Automata as prerequisites. The course learning outcomes are:

CLO #	CLO Description	BT Level	PLO #
CLO 1	Describe the basic phases of compilation and recognize the theories and techniques used in each phase	Understand (BT Level 2)	PLO 2
CLO 2	Analyze and recognize the significance of the several phases through which a typical program is compiled, including a state-of-the-art parsing algorithm (LALR(1)) and a code-optimization algorithm (common subexpression elimination via data-flow analysis)	Analyze (BT Level 4)	PLO 4, PLO 3
CLO 3	Design and implement a lexical analyzer, parser, and intermediate-code generator for a given language, culminating in a semester project	Create (BT Level 6)	PLO 6, PLO 4, PLO 5

Primary Textbooks

These are cited by section number throughout the lecture table below and should be treated as the core references for lecture preparation.

[DB] *Compilers: Principles, Techniques, and Tools* (2nd ed.) by Aho, Lam, Sethi, Ullman — **the "Dragon Book"; primary reference for section numbers below**

[ET] *Engineering a Compiler* (2nd ed.) by Cooper and Torczon

[AP] *Modern Compiler Implementation in C/Java/ML* by Appel

Supplementary / Further Reading

Referenced occasionally for specific topics; useful for deeper treatment but not required for lecture preparation.

[PP] *Programming Language Pragmatics* by Scott

[MU] *Advanced Compiler Design and Implementation* by Muchnick

[AN] *The Definitive ANTLR 4 Reference* by Parr

[GR] *Compilers: A Survey* by Gries

Online / Tool References

[LLVM-LR] LLVM Language Reference Manual
`llvm.org/docs/LangRef.html`

[LLVM-KA] "My First Language Frontend with LLVM" (Kaleidoscope Tutorial)
`llvm.org/docs/tutorial`

[ANTLR] ANTLR4 project site & documentation
`antlr.org`

[TS] Tree-sitter — incremental, error-tolerant parsing library
`tree-sitter.github.io/tree-sitter`

Lecture-Wise Course Outline (30 Lectures)

Note on this revision: following the agreement reached by all instructors, the material after Midterm 1 has been resequenced so that the project's Intermediate-Code-Generator phase can be assigned around Midterm 2. Bottom-up parsing (shift-reduce, LR(0)/SLR, LALR(1)/CLR(1), and parser-generator tool use) now comes *after* Midterm 2 instead of before it; syntax-directed definitions, semantic-analysis foundations (symbol tables), and intermediate code generation now come *before* it. No lecture content has been cut — every topic below is the same one lecture's worth of material as before, only reordered. Lecture count remains exactly 30, and both midterms stay positioned after Lecture 10 and Lecture 20 respectively.

L#	Topic	CLO	Dragon Book [DB]	Other References
Unit 1: Introduction and Motivation (2 Lectures)				
1	Compilers vs. interpreters vs. JIT; why study compilers; applications of compiler technology	CLO1	§1.1, §1.4, §1.5	[ET] §1.1; [AP] Ch.1
2	Structure of a compiler: phases overview; front-/middle-/back-end; preprocessor, assembler, linker, loader	CLO1	§1.2, §2.1–§2.8 (overview)	[PP] §1.5, §1.6
Unit 2: Lexical Analysis (6 Lectures)				
3	Tokens, patterns, lexemes; lexical errors; role of the lexical analyzer	CLO1	§3.1, §3.3	[AP] Ch.2
4	Regular languages vs. non-regular languages; regular expressions and regular definitions, including advanced operators, with extra worked examples	CLO1	§3.3, §3.4	[PP] §3.3
5	Finite automata; transition diagrams; RE → NFA (Thompson's Construction)	CLO1, CLO2	§3.6, §3.7 (Alg. 3.23)	[AP] §2.3–2.4
6	NFA → DFA (subset construction); DFA minimization via transition diagrams; NFA vs. DFA power/performance	CLO1, CLO2	§3.7, §3.9 (Alg. 3.20, 3.36)	[ET] §2.4–2.5
7	Building a lexical analyzer using a lexer-generator tool (hands-on)	CLO3	§3.5, §3.8	[AP] §2.5
Opt.1	***[Optional – instructor's tool choice: Flex]*** Building the L7 lexer concretely with Flex. *Not part of Midterm 1 – assessed via the project.*	CLO3	(tool choice for §3.5, §3.8)	—
Unit 3: Context-Free Grammars (Lectures 8–9)				
8	Context-free grammars; derivations; parse trees; ambiguity	CLO1	§4.1–§4.2, §4.3.3	[GR] Ch.4
9	Grammar engineering I: eliminating left recursion and left factoring	CLO1, CLO2	§4.3	[ET] §3.5

L#	Topic	CLO	Dragon Book [DB]	Other References
Unit 2 (cont.): Lexical Analyzer Construction (Lecture 10)				
10	Input buffering: two-buffer scheme, `lexeme_begin` and `forward` pointers; recognition of tokens via transition diagrams and transition tables; hand-coding a lexical analyzer from a transition diagram	CLO1, CLO3	§3.2, §3.4	[AP] §2.2
— Midterm 1 (after Lecture 10; covers L1–L10) —				
Unit 3 (cont.): Context-Free Grammars & Top-Down Parsing (Lecture 11)				
11	Grammar engineering II: removing ambiguity (e.g. dangling-else), disambiguation techniques, non-CFG constructs; top-down parsing overview and recursive-descent parsing	CLO1, CLO2, CLO3	§4.3.3, §4.3, §4.4.1	[GR] Ch.4; [AP] §3.2
Unit 4: Top-Down Parsing Algorithms (Lectures 12–13)				
12	FIRST and FOLLOW sets; LL(1) parsing-table construction	CLO1, CLO2	§4.4.2–§4.4.4	[ET] §3.3
13	LL(1) predictive-parsing algorithm; panic-mode error recovery	CLO2, CLO3	§4.4.3, §4.4.5–§4.4.9	[PP] §3.4.5
Unit 5: Syntax-Directed Definitions & Semantic-Analysis Foundations (Lectures 14–16)				
14	Syntax-directed definitions; S-attributed and L-attributed grammars	CLO1	§5.1–§5.2	[ET] Ch.4 intro
15	Syntax trees; SDT schemes; introduction to type checking	CLO1, CLO2	§5.3–§5.5, §6.5	[PP] §6.1–6.2
16	Symbol tables, scope resolution, static semantic checking, with emphasis on practical implementation	CLO1, CLO2	§2.7, §6.3	[AP] Ch.5
Unit 6: Intermediate Code Generation (Lectures 17–20)				
17	Intermediate representations: DAGs, three-address code, quadruples/triples	CLO1, CLO3	§6.1–§6.2	[ET] §5.1–5.3
18	Translating expressions and control-flow statements; backpatching	CLO2, CLO3	§6.4, §6.6–§6.7	[AP] §7.2–7.3
19	Type checking during translation; intermediate code for procedure calls	CLO1, CLO2	§6.3, §6.5, §6.9	[AP] §5.2, §7.4
20	Building an intermediate-code generator: full AST → three-address-code translator *(hands-on)*	CLO3	§6.1–§6.9 (synthesis)	[ET] Ch.5
Opt.5	***[Optional – instructor's tool choice: LLVM, Part 1]*** Architecture, LLVM IR, key classes (Module, Function, BasicBlock, IRBuilder). Others may substitute GCC internals, a custom bytecode VM, or skip and use the time as buffer/review.	CLO3	(tool choice for §6.1–§6.9)	[LLVM-LR] LLVM Language Reference Manual
Opt.6	***[Optional – instructor's tool choice: LLVM, Part 2]*** Generating LLVM IR from an AST, end to end (hands-on).	CLO3	(tool choice for §6.1–§6.9)	[LLVM-KA] LLVM Kaleidoscope Tutorial
— Midterm 2 (after Lecture 20; covers L11–L20: top-down parsing, SDD & semantic-analysis foundations, intermediate code generation) —				
Unit 7: Bottom-Up Parsing Algorithms (Lectures 21–25)				
21	Bottom-up parsing: shift-reduce parsing, handles, shift-reduce & reduce-reduce conflicts	CLO1, CLO2	§4.5	[AP] §3.3
22	LR(0) items & automaton; SLR parsing	CLO2	§4.6	[ET] §3.4.1–3.4.2
23	**LALR(1) parsing** — the industry-standard, state-of-the-art table-driven algorithm (used by Yacc/Bison), with a brief comparative look at canonical LR(1)	CLO2	§4.7.1–§4.7.5 (canonical LR(1) covered briefly; LALR(1) in full)	[ET] §3.4.3–3.4.4; [AN] Ch.1

L#	Topic	CLO	Dragon Book [DB]	Other References
24	Using ambiguous grammars in practice; building a parser with a parser-generator tool (hands-on)	CLO3	§4.8, §4.9	[AN] Ch.4–5
Opt.2	***[Optional – instructor's tool choice: Bison/Yacc]*** Building the L24 parser concretely with Bison/Yacc.	CLO3	(tool choice for §4.8, §4.9)	—
Opt.3	***[Optional – modern alternative]*** ANTLR4: generates a combined lexer + adaptive LL(*) parser from one grammar file, with strong IDE/multi-language support – a direct modern successor to Lex/Yacc.	CLO3	(not in DB; extends §4.9)	[ANTLR] ANTLR4 site
Opt.4	***[Optional – modern alternative]*** Tree-sitter: incremental, error-tolerant GLR-style parsing – the current state of the art for editor/IDE-grade parsing (used by Neovim, GitHub, VS Code).	CLO3	(not in DB; extends §4.9)	[TS] Tree-sitter site
25	**Review 3:** bottom-up parsing algorithms (shift-reduce, LR(0)/SLR, LALR(1), canonical LR(1)) – consolidation and worked examples (no new material; top-down parsing and SDT were already reviewed as part of Midterm 2 preparation)	CLO1, CLO2	(review of §4.5–4.9)	—
Unit 8: Run-Time Environments (2 Lectures)				
26	Storage organization; stack allocation; activation records; nonlocal access	CLO1	§7.1–§7.3	[ET] Ch.6
27	Heap management; introduction to garbage collection	CLO1	§7.4–§7.5	[AP] Ch.13
Unit 9: Code Generation & Optimization (3 Lectures)				
28	Basic blocks & flow graphs; local optimizations (constant folding, copy/constant propagation, CSE); peephole optimization	CLO1, CLO2	§8.4–§8.5, §8.7	[MU] Ch.12
29	**Optimization Algorithm:** Common Subexpression Elimination via Available-Expressions data-flow analysis	CLO2	§9.1–§9.3	[MU] §13.1–13.2
30	Register allocation & assignment; instruction selection overview *(previously-uncovered material from Ch. 8, now restored)*	CLO1, CLO2	§8.8–§8.9	[AP] Ch.11
— Comprehensive Final Exam (covers L1–L30, emphasis on L21–L30) —				

Open item for the team: the project bullet below originally aligned the lexer/parser/IR-generator project phases with the hands-on lectures in that order (L7, old-L17, old-L25). Under this revision, the hands-on IR-generator lecture (new L20) now comes *before* the hands-on parser-generator-tool lecture (new L24). If the "Intermediate Code Generator" project phase is to be assigned around Midterm 2 as intended, the project's parser phase likely needs to rely on the hand-written recursive-descent parser from L11–L13 (available well before Midterm 2) rather than waiting for the tool-based parser at L24. Please confirm how the project phases should be sequenced so the assessment instruments section and project handout can be updated consistently.

Assessment Instruments

- **Project** – students build a working compiler (lexer → parser → IR generator, optionally with basic optimization) for a small language, in stages aligned with the hands-on lectures (L7 lexer; L20 IR generator; L24 parser-generator tool – **see the open item above regarding project-phase sequencing**).

- **Quizzes** – 7 short quizzes distributed across the semester (roughly one every 4 lectures), primarily on the lecture immediately preceding each quiz.
- **Assignments** – 3–4 assignments: (1) regular expressions & finite automata, (2) grammar transformation & top-down parsing, (3) intermediate code generation (now timed around L17–L20), and optionally (4) bottom-up parsing tables (SLR/CLR/LALR) / a chosen optimization (now timed after Midterm 2).
- **Midterm 1** (after L10), **Midterm 2** (after L20), **Comprehensive Final** (after L30).

Tentative Grading Scheme

- Project – 10 %
- Assignments – 5 %
- Quizzes – 15 %
- Midterms – 30 %
- Final Exam – 40 %